If the join table for a has_and_belongs_to_many association has additional columns beyond the two foreign keys, these columns will be added as attributes to records retrieved via that association. Records returned with additional attributes will always be read-only, because Rails cannot save changes to those attributes.
The use of extra attributes on the join table in a has_and_belongs_to_many association is deprecated. If you require this sort of complex behavior on the table that joins two models in a many-to-many relationship, you should use a has_many :through association instead of has_and_belongs_to_many.

4.4.1.2 collection(force_reload = false)

The collection method returns an array of all of the associated objects. If there are no associated objects, it returns an empty array.

```
@assemblies = @part.assemblies
```

4.4.1.3 collection<<(object, ...)

The collection<< method adds one or more objects to the collection by creating records in the join table.

```
@part.assemblies << @assembly1
```

This method is aliased as collection.concat and collection.push.

4.4.1.4 collection.delete(object, ...)

The collection.delete method removes one or more objects from the collection by deleting records in the join table. This does not destroy the objects.

```
@part.assemblies.delete(@assembly1)
```

This does not trigger callbacks on the join records.

4.4.1.5 collection.destroy(object, ...)

The collection.destroy method removes one or more objects from the collection by running destroy on each record in the join table, including running callbacks. This does not destroy the objects.

```
@part.assemblies.destroy(@assembly1)
```

4.4.1.6 collection=(objects)

The collection= method makes the collection contain only the supplied objects, by adding and deleting as appropriate.

4.4.1.7 collection_singular_ids

The collection_singular_ids method returns an array of the ids of the objects in the collection.

```
@assembly_ids = @part.assembly_ids
```

4.4.1.8 collection_singular_ids=(ids)

The collection_singular_ids= method makes the collection contain only the objects identified by the supplied primary key values, by adding and deleting as appropriate.

4.4.1.9 collection.clear

The collection.clear method removes every object from the collection by deleting the rows from the joining table. This does not destroy the associated objects.

4.4.1.10 collection.empty?

The collection.empty? method returns true if the collection does not contain any associated objects.

```
<% if @part.assemblies.empty? %>
  This part is not used in any assemblies
<% end %>
```

4.4.1.11 collection.size

The collection.size method returns the number of objects in the collection.

```
@assembly_count = @part.assemblies.size
```

4.4.1.12 collection.find(...)

The collection.find method finds objects within the collection. It uses the same syntax and options as ActiveRecord::Base.find. It also adds the additional condition that the object must be in the collection.